

Week 09

Database Applications

Cảm ơn thầy Trần Duy Quang đã cung cấp template cho môn học
Cảm ơn thầy Hồ Tuấn Thanh và thầy Đỗ Nguyên Kha đã xây dựng nội dung thực hành



Lecturer: **TS. Lê Thị Nhân.**
Lab TA: **ThS. Nguyễn Hải Đăng.**

1

Notes

Put **ALL** source code (include SQL files) into a zip/rar 2012xxx_yy.zip/rar (yy is the number fulfilled requirements):

- Comment Student ID and Full name on top of file
- Run ALL lines in the file without any error
- USE the RIGHT database
- Clean **obj** directory

2 Lab

Download and install (Windows/Mac/Linux):

- .NET Core SDK (current version 2.1): <https://www.microsoft.com/net/download>
- Visual Studio Code: <https://code.visualstudio.com/>

Create console application

Open Terminal (Mac/Linux/Windows) and run the commands:

```
mkdir cs386
cd cs386
mkdir ConsoleApp
cd ConsoleApp
dotnet new console
```

Those commands create a new .NET Core Console Application in the **ConsoleApp** directory.

Try to run application:

```
dotnet run
```

It should say “Hello World!” message on the screen. You can also create a .NET **Core** Console Application on the newest Visual Studio Community but it is not mentioned here because it is quite easy to use the UI.

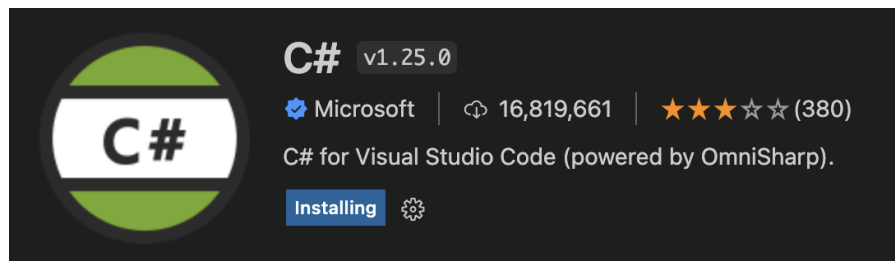
```
1. khado@Khas-Macbook-Pro: ~/Downloads/cs386/consoleapp (zsh)
khado@Khas-Macbook-Pro ~/Downloads/cs386$ ls
khado@Khas-Macbook-Pro ~/Downloads/cs386$ mkdir consoleapp
khado@Khas-Macbook-Pro ~/Downloads/cs386$ cd consoleapp
khado@Khas-Macbook-Pro ~/Downloads/cs386/consoleapp$ dotnet new console

The template "Console Application" was created successfully.

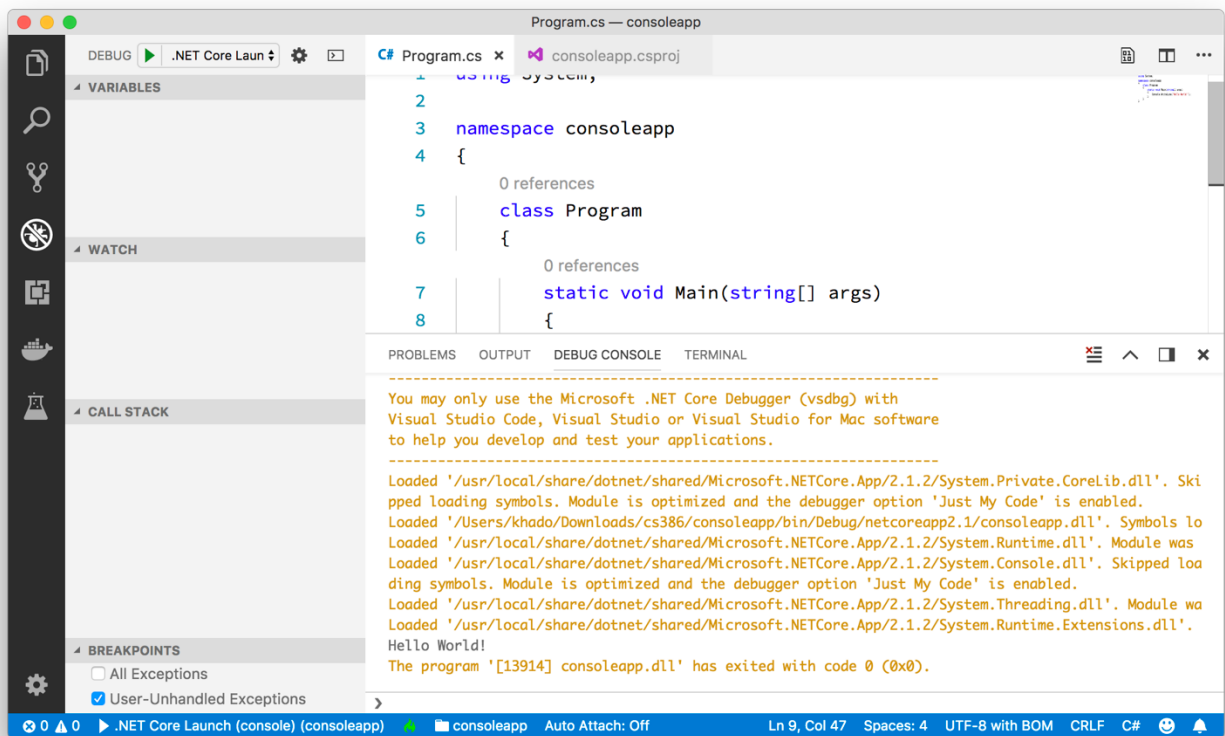
Processing post-creation actions...
Running 'dotnet restore' on /Users/khado/Downloads/cs386/consoleapp/consoleapp.csproj...
  Restoring packages for /Users/khado/Downloads/cs386/consoleapp/consoleapp.csproj...
    Generating MSBuild file /Users/khado/Downloads/cs386/consoleapp/obj/consoleapp.csproj.nu
    get.g.props.
    Generating MSBuild file /Users/khado/Downloads/cs386/consoleapp/obj/consoleapp.csproj.nu
    get.g.targets.
    Restore completed in 285.72 ms for /Users/khado/Downloads/cs386/consoleapp/consoleapp.cs
    proj.
  Restore succeeded.

khado@Khas-Macbook-Pro ~/Downloads/cs386/consoleapp$ ls
Program.cs  consoleapp.csproj  obj
khado@Khas-Macbook-Pro ~/Downloads/cs386/consoleapp$ dotnet run
Hello World!
khado@Khas-Macbook-Pro ~/Downloads/cs386/consoleapp$
```

Open source code in the Visual Studio Code (VS Code), you will be suggested to install additional package to support C# application, please follow its recommendation. Remember to reload/restart VS Code after the package was installed.



VS Code will ask to create configuration file for C# Application, please also allow it to be done. Try to run the application inside VS Code:



Then back to the terminal to install supporting package for connecting to database by NuGet:

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Relational
```

Create an simple version of **University** database:

```
CREATE DATABASE University
GO
USE University
CREATE TABLE Students (
    ID int IDENTITY(1,1) PRIMARY KEY,
    Name nvarchar(50) NOT NULL
)
INSERT INTO Students (Name) VALUES (N'Harry Potter')
INSERT INTO Students (Name) VALUES (N'Ron Weasley')
INSERT INTO Students (Name) VALUES (N'Hermione Granger')
```

The **IDENTITY** modifier will generate ID automatically.

Back to VS Code and create a new file **Student.cs**:

```
using System.ComponentModel.DataAnnotations;

namespace ConsoleApp
{
    public class Student
    {
        public int ID { get; set; }
        [StringLength(50)]
        public string Name { get; set; }
    }
}
```

Create **UniversityDbContext.cs** to connect to SQL Server database:

```
using Microsoft.EntityFrameworkCore;

namespace ConsoleApp
{
    public class UniversityDbContext: DbContext
    {
        public DbSet<Student> Students { get; set; }
        protected override void OnConfiguring(DbContextOptionsBuilder
optionsBuilder)
        {
            optionsBuilder.UseSqlServer(@"Server=localhost;Database=University;User
Id=sa;Password=P@ssw0rd;");
        }
    }
}
```

Replace **localhost** by docker-machine IP if you are using Docker Toolbox on the old Windows version. This information should be the same as SQL Server Operations Studio database connection.

Change the **Program.cs** to read student information from database and print to console:

```
using System;
using System.Linq;
using Microsoft.EntityFrameworkCore;

namespace ConsoleApp
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
            var context = new UniversityDbContext();
            var students = context.Students.FromSqlRaw("SELECT * FROM
Students").ToList();
            foreach (var student in students)
            {
                Console.WriteLine(String.Format("ID: {0}, Name: {1}", student.ID,
student.Name));
            }
        }
    }
}
```

Try to insert new student into database:

```
using System;
using System.Linq;
using Microsoft.EntityFrameworkCore;
using System.Data.SqlClient;

namespace ConsoleApp
{

```

```
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Hello World!");
        var context = new UniversityDbContext();
        Console.WriteLine("Enter a student name: ");
        // Insert new student
        var name = Console.ReadLine();
        context.Database.ExecuteSqlRaw("INSERT INTO Students (Name) VALUES (@Name)", new SqlParameter("@Name", name));
        // Print all students
        var students = context.Students.FromSqlRaw("SELECT * FROM Students").ToList();
        foreach (var student in students)
        {
            Console.WriteLine(String.Format("ID: {0}, Name: {1}", student.ID, student.Name));
        }
    }
}
```

To use the interactive terminal:

1. Open `.vscode/launch.json`.
2. Change the `console` setting from `internalConsole` to `integratedTerminal`:
3. Save your changes.

You can also use Entities Framework and LINQ syntax but this lab is focused on SQL Query.

Create a web application

Run the command to create a new ASP.NET Core web application:

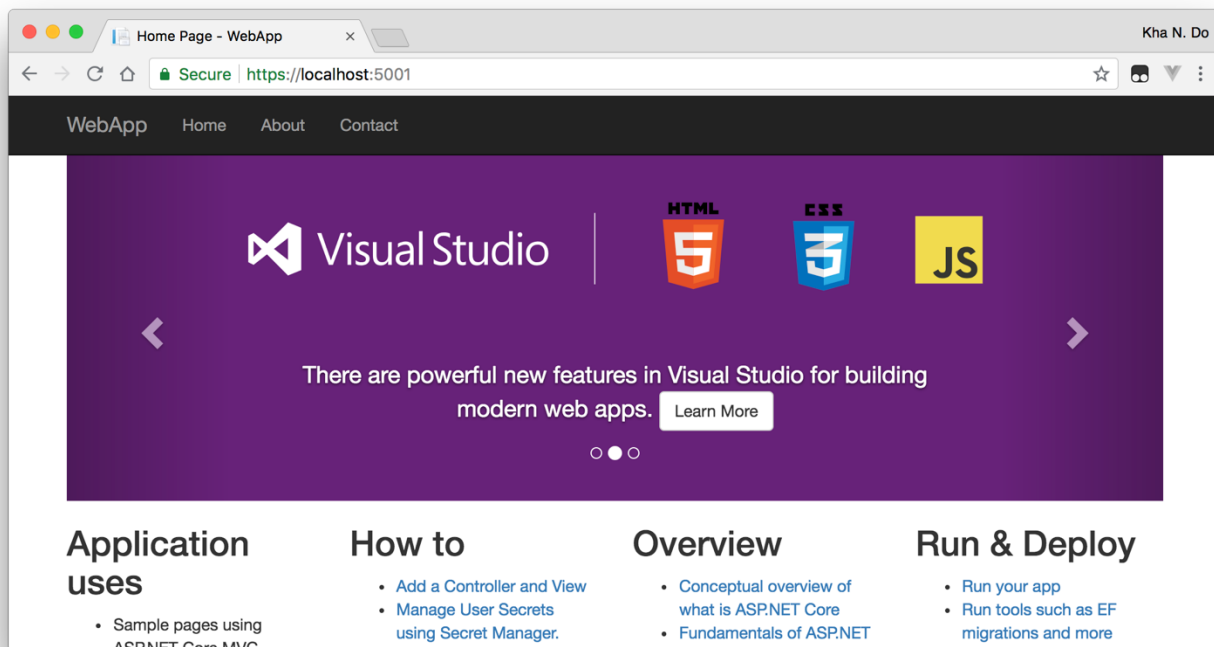
```
mkdir WebApp
```



```
cd WebApp
dotnet new mvc
dotnet dev-certs https --trust
dotnet run
```

Open <https://localhost:5001/> in a browser (Chrome, Firefox, Safari...)

Add



Add supporting packages for database

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Relational
```

Add **Student.cs** and **UniversityDbContext.cs**

```
using System.ComponentModel.DataAnnotations;

namespace WebApp.Models
{
    public class Student
    {

```

```
        public int ID { get; set; }
        [StringLength(50)]
        public string Name { get; set; }
    }
}
```

```
using Microsoft.EntityFrameworkCore;

namespace WebApp.Models
{
    public class UniversityDbContext : DbContext
    {
        public UniversityDbContext(DbContextOptions<UniversityDbContext> options) :
base(options)
        {}
        public DbSet<Student> Students { get; set; }
    }
}
```

Add dependency injection in **Startup.cs**, find **AddMvc** and add database context after it

```
        services
            .AddMvc()
            .SetCompatibilityVersion(CompatibilityVersion.Version_2_1);
        var connectionString = @"Server=localhost;Database=University;User
Id=sa;Password=P@ssw0rd;";
        services
            .AddDbContext<UniversityDbContext>(options =>
options.UseSqlServer(connectionString));
```

Create Controller for display student list **Controllers/StudentController.cs**

```
using System;
using System.Linq;
using Microsoft.AspNetCore.Mvc;
```

```
using Microsoft.EntityFrameworkCore;
using WebApp.Models;

namespace WebApp.Controllers
{
    public class StudentController : Controller
    {
        private readonly UniversityDbContext _context;
        public StudentController(UniversityDbContext context)
        {
            _context = context;
        }
        public IActionResult Index()
        {
            var students = _context.Students.FromSql("SELECT * FROM
Students").ToList();
            return View(students);
        }
    }
}
```

And **Views/Student/Index.cshtml**

```
@{
    ViewData["Title"] = "Student List";
}
<h2>@ViewData["Title"]</h2>

<div class="row">
    <table class="table table-striped table-bordered table-hover">
        <thead>
            <tr>
                <th>ID</th>
```

```
        <td>Name</td>
    </tr>
</thead>
<tbody>
    @foreach (var student in Model) {
        <tr>
            <td>@student.ID</td>
            <td>@student.Name</td>
        </tr>
    }
</tbody>
</table>
</div>
```

Then open <https://localhost:5001/Student> in a browser

The source code of the application can be found at: <https://github.com/nguyenkha/cs386>. You should do step by step to understand how the application works.

3 Homework

Extend the Lab with additional student information:

- ID int auto_increment
- StudentID varchar(7) unique
- LastName nvarchar(50)
- FirstName nvarchar(50)
- Gender char(1)
- Class nvarchar(20)

Console Application:

- Display menu
- List all student
- Add a new student
- Remove a student
- Update a student

Web Application:

- List all student